

Observability und Diagnostics

Übersicht

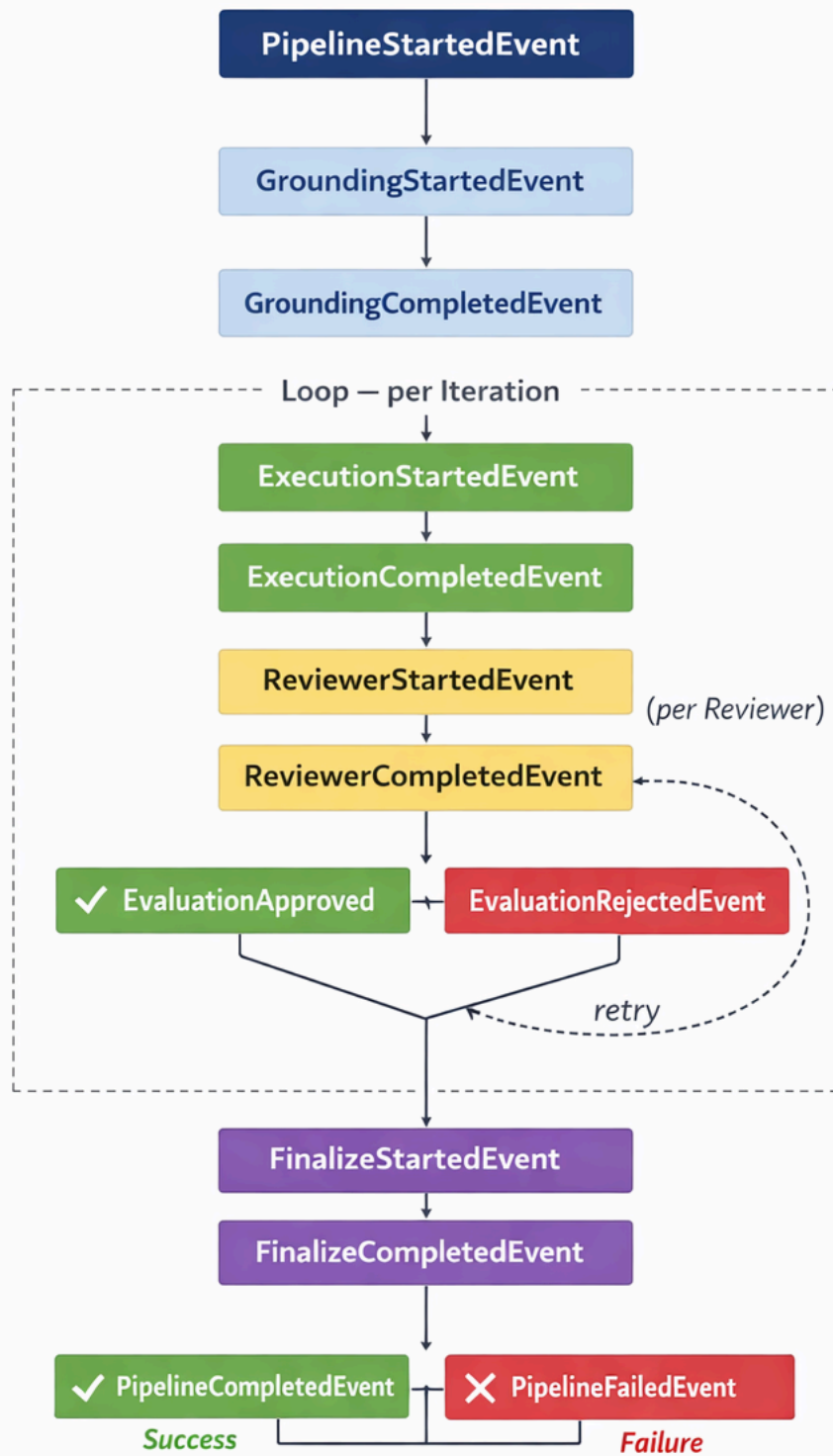
Geef.Sdk bietet drei Ebenen der Beobachtbarkeit:

1. **Structured Events** — Typisierte Events über `IGeefEventSink`
2. **Distributed Tracing** — OpenTelemetry-kompatible Spans via `ActivitySource`
3. **Logging** — Strukturiertes Logging über `ILogger`

Event-System

13 Event-Typen

Das SDK emittiert Events an jedem Übergang im Pipeline-Lifecycle:



DelegateEventSink — Event-Hooks

Für schnelles Prototyping und einfache Szenarien. Bietet Hooks für 11 der 13 Event-Typen — `ReviewerStartedEvent` und `ReviewerCompletedEvent` sind nur über eine eigene `IGeefEventSink`-Implementierung zugänglich.

```

.ConfigureEvents(e =>
{
    e.OnPipelineStarted = evt =>
    {
        Console.WriteLine($"[{evt.Timestamp:HH:mm:ss}] Start: {evt.Input}");
        return Task.CompletedTask;
    };

    e.OnExecutionCompleted = evt =>
    {
        Console.WriteLine($" Iteration {evt.Iteration}: Execution abgeschlossen ({ev
t.Duration.TotalMilliseconds:F0} ms)");
        return Task.CompletedTask;
    };

    e.OnEvaluationRejected = evt =>
    {
        var findings = evt.Aggregate.AllFindings;
        Console.WriteLine($" Iteration {evt.Iteration}: {findings.Count} Finding
(s)");
        foreach (var f in findings)
            Console.WriteLine($" [{f.Severity}] {f.ReviewerName}: {f.Message}");
        return Task.CompletedTask;
    };

    e.OnPipelineCompleted = evt =>
    {
        Console.WriteLine($"Fertig: {evt.TotalIterations} Iterationen in {evt.TotalDu
ration.TotalSeconds:F1}s");
        return Task.CompletedTask;
    };

    e.OnPipelineFailed = evt =>
    {
        Console.WriteLine($"FEHLGESCHLAGEN: {evt.Reason} nach {evt.TotalIterations} I
terationen");
        return Task.CompletedTask;
    };
})

```

LoggingEventSink — Strukturiertes Logging

Loggt alle Events über `Microsoft.Extensions.Logging.ILogger`:

```
var logger = loggerFactory.CreateLogger("GeefPipeline");  
    .AddEventSink(new LoggingEventSink(logger))
```

Beispiel-Ausgabe:

```
info: GeefPipeline[0] Pipeline started: RunId=alb2c3d4e5f6, Input="Generiere Unittes  
t..."  
info: GeefPipeline[0] Grounding started: RunId=alb2c3d4e5f6  
info: GeefPipeline[0] Grounding completed: RunId=alb2c3d4e5f6, Duration=00:00:01.234  
info: GeefPipeline[0] Execution started: RunId=alb2c3d4e5f6, Iteration=1  
info: GeefPipeline[0] Execution completed: RunId=alb2c3d4e5f6, Iteration=1, Duration=  
00:00:05.678  
info: GeefPipeline[0] Reviewer started: RunId=alb2c3d4e5f6, Reviewer=SecurityCheck  
info: GeefPipeline[0] Reviewer completed: RunId=alb2c3d4e5f6, Reviewer=SecurityCheck,  
Decision=Approved  
warn: GeefPipeline[0] Evaluation rejected: RunId=alb2c3d4e5f6, Iteration=1, Decision=  
Continue  
info: GeefPipeline[0] Pipeline completed: RunId=alb2c3d4e5f6, Iterations=3, Duration=  
00:00:18.901
```

Eigener EventSink

```
public class MetricsEventSink : IGeefEventSink
{
    private readonly IMetrics _metrics;

    public MetricsEventSink(IMetrics metrics) => _metrics = metrics;

    public ValueTask PublishAsync(IGeefEvent geefEvent, CancellationToken ct = default)
    {
        switch (geefEvent)
        {
            case PipelineCompletedEvent e:
                _metrics.RecordPipelineDuration(e.TotalDuration);
                _metrics.IncrementIterationCount(e.TotalIterations);
                break;
            case PipelineFailedEvent e:
                _metrics.IncrementFailureCount(e.Reason.ToString());
                break;
            case ReviewerCompletedEvent e:
                _metrics.RecordReviewDuration(e.Result.ReviewerName, e.Result.Duration);
                break;
        }
        return ValueTask.CompletedTask;
    }
}
```

OpenTelemetry Tracing

ActivitySource

Das SDK verwendet eine zentrale `ActivitySource`:

```
GeefDiagnostics.ActivitySource // Name: "Geef.Sdk", Version: "1.0.0"
```

Span-Hierarchie



Tags

Jeder Span enthält:

- `geef.run_id` — Korrelations-ID
- `geef.iteration` — Iterationsnummer (wo anwendbar)
- `geef.phase` — Aktuelle Phase (wenn TracingMiddleware aktiv)

Integration

```
// In Program.cs
builder.Services.AddOpenTelemetry()
    .WithTracing(tracing =>
    {
        tracing.AddSource("Geef.Sdk"); // GEEF-Spans erfassen
        tracing.AddOtlpExporter(o => o.Endpoint = new Uri("http://localhost:4317"));
    });
```

RunId — Korrelation

Jeder Pipeline-Lauf erhält eine eindeutige 12-Zeichen-RunId:

- Enthalten in allen Events (`IGeefEvent.RunId`)

- Enthalten in allen Exceptions (`GeefException.RunId`)
- Enthalten in allen Tracing-Spans (Tag `geef.run_id`)
- Enthalten im Ergebnis (`GeefPipelineResult.RunId`)

Damit können alle Logs, Traces und Metriken eines Laufs korreliert werden.

Debugging-Tipps

Pipeline-Verlauf inspizieren

```
var result = await pipeline.RunAsync("input");

foreach (var record in result.History.Records)
{
    Console.WriteLine($"Iteration {record.Iteration}:");
    Console.WriteLine($"  Execution-Dauer: {record.ExecutionDuration}");
    Console.WriteLine($"  Findings: {record.FindingFingerprints.Count}");
    foreach (var review in record.EvaluationResult.Reviews)
        Console.WriteLine($"    {review.ReviewerName}: {review.Decision} ({review.FindingFingerprints.Count} Findings)");
}
```

Context zu jedem Zeitpunkt einsehen

Da der Context immutabel ist, können Sie ihn in Events oder Middleware inspizieren:

```
.ConfigureEvents(e =>
{
    e.OnExecutionCompleted = evt =>
    {
        // Context nach Execution einsehen (nicht möglich, da Event keinen Context enthält)

        // Stattdessen: Middleware verwenden
        return Task.CompletedTask;
    };
})

.UseMiddleware(new ContextInspectorMiddleware())
```



```
public class ContextInspectorMiddleware : IGeefMiddleware
{
    public async Task InvokeAsync(GeefMiddlewareContext ctx, Func<Task> next)
    {
        Console.WriteLine($"[{ctx.Phase}] Before - Keys: {string.Join(", ", ctx.RunContext.Keys)}");
        await next();
        Console.WriteLine($"[{ctx.Phase}] After");
    }
}
```